# Detecting Docker Socket Abuse and Container Breakout using auditd and Splunk

## Overview

In this lab, I simulated a realistic container escape scenario and built detections around it using Splunk and auditd.

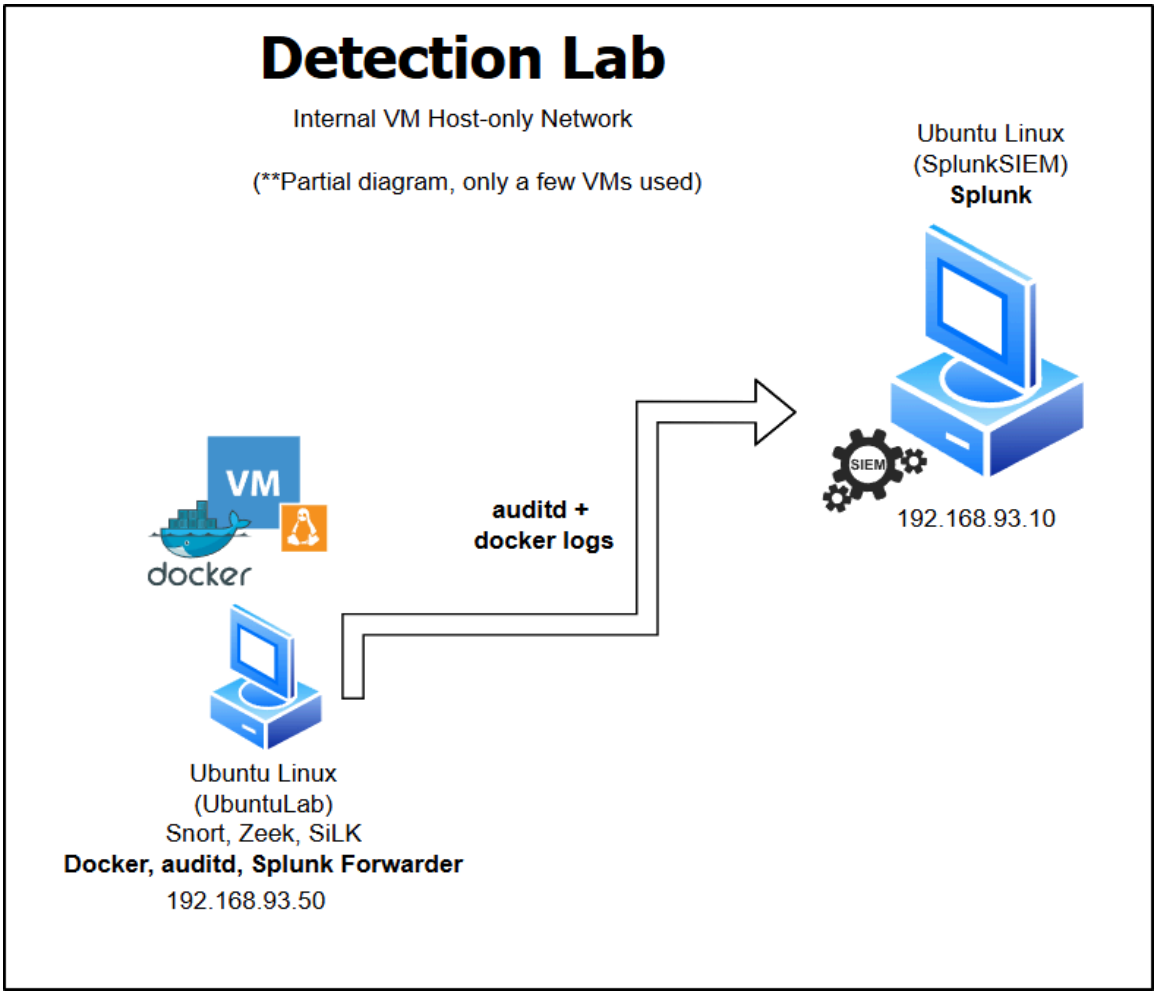
The focus wasn't just on "getting logs," it was more about determining what we as defenders can realistically observe when it comes to docker container exploitation.

What started as a simple Docker logging exercise quickly turned into something more interesting - a comparison of telemetry sources, and a lesson about detection signals and visibility.

AI assistance was used for documentation support; all lab work, testing, and validation were performed independently.

---

## Lab Environment



- UbuntuLab (Docker host)
- Docker Engine
- SplunkSIEM (SIEM)
- Splunk Universal Forwarder
- auditd (host-based process telemetry)
- Zeek (network telemetry, limited visibility in this setup)
- VirtualBox-based lab network

---

## Attack Scenario

The simulated attack followed a realistic progression:

1. **Initial container access**
   - Attacker gains shell access into a running container

```
root@ubuntulab:/home/labuser/Desktop# docker run -dit --name attacker-container ubuntu:24.04 bash
ef23ac831cae6f521ce6f29daeec749c56878d46e3fa6537a5effd3ff0d39cc8
root@ubuntulab:/home/labuser/Desktop# docker exec -it attacker-container bash
```

*Attacker creates and gains interactive access to a Docker container on the host system.*

2. **Tooling and reconnaissance**
   - Installs utilities (`curl`, `dnsutils`)
   - Performs DNS lookups and HTTP requests

```
root@ef23ac831cae:/# apt install -y curl dnsutils iputils-ping
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bind9-dnsutils bind9-host bind9-libs ca-certificates krb5-locales libbrotli1 li
  libicu74 libjson-c5 libk5crypto3 libkeyutils1 libkrb5-3 libkrb5support0 libldap
  libpam-cap libpsl5t64 librtmp1 libsasl2-2 libsasl2-modules libsasl2-modules-db
Suggested packages:
  krb5-doc krb5-user mmdb-bin libsasl2-modules-gssapi-mit | libsasl2-modules-gssa
  libsasl2-modules-sql
The following NEW packages will be installed:
  bind9-dnsutils bind9-host bind9-libs ca-certificates curl dnsutils iputils-ping
  libedit2 libgssapi-krb5-2 libicu74 libjson-c5 libk5crypto3 libkeyutils1 libkrb5
  libmaxminddb0 libnghttp2-14 libpam-cap libpsl5t64 librtmp1 libsasl2-2 libsasl2-
  openssl publicsuffix
The following packages will be upgraded:
```

*Attacker installs common reconnaissance tools inside the container to enable network enumeration and outbound communication.*

```
root@ef23ac831cae:/# nslookup example.com
Server:         75.75.75.75
Address:        75.75.75.75#53

Non-authoritative answer:
Name:   example.com
Address: 172.66.147.243
Name:   example.com
Address: 104.20.23.154
Name:   example.com
Address: 2606:4700:10::6814:179a
Name:   example.com
Address: 2606:4700:10::ac42:93f3

root@ef23ac831cae:/# curl http://example.com
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="v
body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-ser
348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use
e in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a
```

*Basic reconnaissance is performed from within the container, including DNS resolution and outbound HTTP requests.*

3. **Misconfiguration abuse**
   - Launches a container with Docker socket mounted:

     -v /var/run/docker.sock:/var/run/docker.sock

```
root@ubuntulab:/home/labuser/Desktop# docker run -dit \
  --name attacker-priv \
  -v /var/run/docker.sock:/var/run/docker.sock \
  ubuntu:24.04 bash
e4c3510f0cb3b46128cbab9f341d606ae4ac1d737ec73fd91420d701fb97aed0
root@ubuntulab:/home/labuser/Desktop#
```

*A new container is launched with the Docker socket mounted, exposing the host Docker daemon to the container.*

4. **Host control via Docker**
   - Executes Docker commands from inside the container
   - Launches new containers from the host daemon

```
root@e4c3510f0cb3:/# docker ps
CONTAINER ID   IMAGE          COMMAND   CREATED              STATUS
e4c3510f0cb3   ubuntu:24.04   "bash"    About a minute ago   Up About a minute
ef23ac831cae   ubuntu:24.04   "bash"    4 minutes ago        Up 4 minutes
e827d964dc5c   ubuntu:24.04   "bash"    7 hours ago          Up 7 hours
```

*Attacker interacts with the host Docker daemon from within the container, confirming visibility and control over running containers.*

```
root@e4c3510f0cb3:/# apt install -y docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```

*Docker tooling is installed inside the container to facilitate direct interaction with the host Docker daemon..*

5. **Breakout behavior**
   - Mounts host filesystem:

     -v /:/host

   - Runs privileged container:

     --privileged

```
root@e4c3510f0cb3:/# docker ps
CONTAINER ID   IMAGE          COMMAND   CREATED              STATUS
e4c3510f0cb3   ubuntu:24.04   "bash"    About a minute ago   Up About a minute
ef23ac831cae   ubuntu:24.04   "bash"    4 minutes ago        Up 4 minutes
e827d964dc5c   ubuntu:24.04   "bash"    7 hours ago          Up 7 hours
root@e4c3510f0cb3:/# docker run --rm -it -v /:/host ubuntu bash
root@20d70209402a:/# ls /host
alert_json.txt  bin.usr-is-merged  cdrom  dev  home  lib.usr-is-merged  lost+fo
bin             boot               data   etc  lib   lib64              media
```

*Attacker mounts the host filesystem into a container, effectively gaining access to the underlying host system.*

```
root@ubuntulab:/home/labuser/Desktop# docker exec -it attacker-container bash
root@ef23ac831cae:/# whoami
root
```

*Attacker re-enters the container environment to validate access and maintain interactive control.*

At this point, the attacker has effectively transitioned from a container foothold to **host-level control**.

---

# Data Collection

### Docker Logs (Initial Attempt)

Docker container logs were first ingested into Splunk via:

/var/lib/docker/containers/*/*-json.log

These logs capture container `stdout/stderr`, which is useful for visibility into application output. However, - they do not reliably capture the actual commands being executed. This made them insufficient for detecting meaningful attacker behavior.

---

### auditd (Primary Detection Source)

To improve visibility, I implemented `auditd` on the host and created rules to monitor Docker execution:

auditctl -a always,exit -F arch=b64 -S execve -F exe=/usr/bin/docker -k docker_cmd

This allowed capture of:

- Docker command execution (`execve`)
- Command arguments (`a0`, `a1`, etc.)
- High-fidelity host activity

This became the primary detection source.

---

# Detection Challenges

Initially, I attempted to build detections using parsed fields like `cmdline` in Splunk.

These searches failed, because:

- Splunk wasn't consistently extracting those fields
- The data existed only in raw audit logs

The solution involved using pattern matching and direct string searches against `EXECVE` records.

---

# Splunk Detection Logic

All SPL detections are based on:

```
index=main sourcetype=linux:audit
"key=\"docker_cmd\"" "type=EXECVE"
```

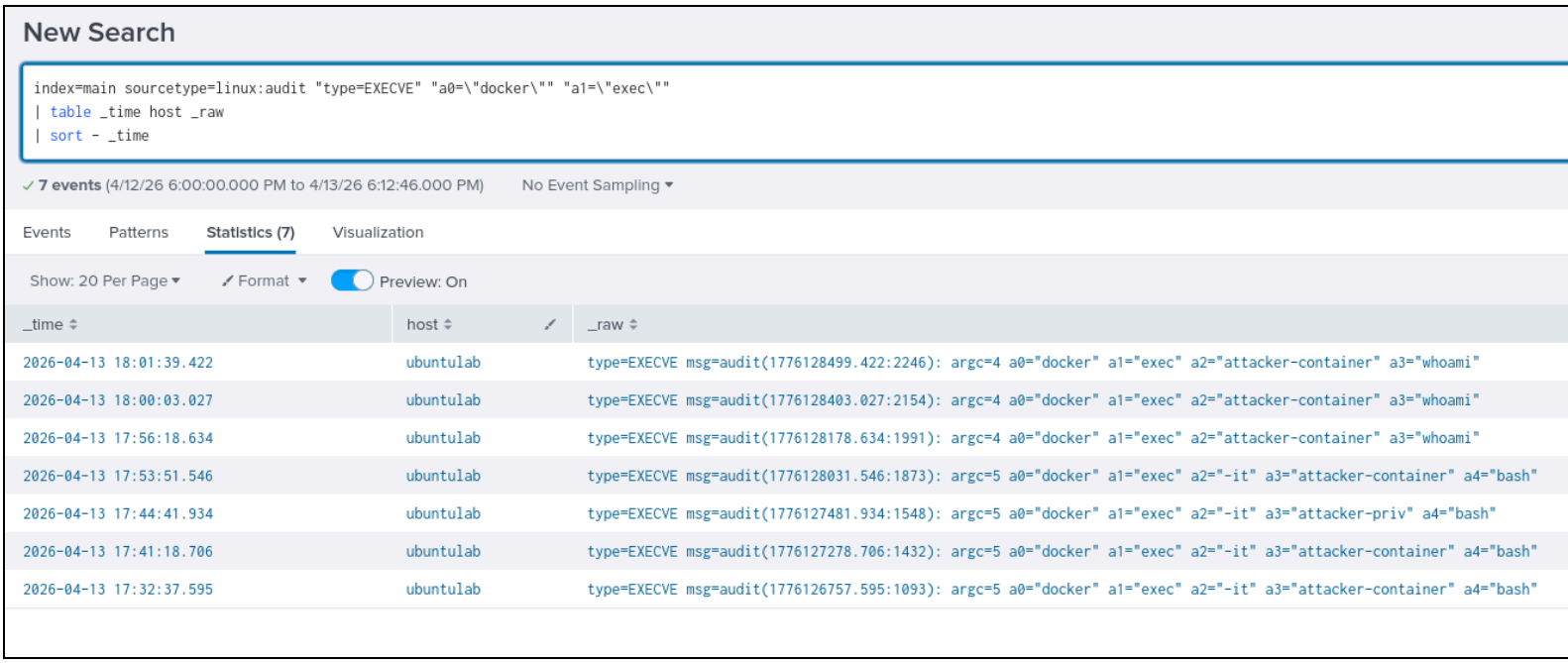
---

### Interactive Container Access

```
index=main sourcetype=linux:audit
"key=\"docker_cmd\"" "type=EXECVE" "a0=\"docker\""
"a1=\"exec\""
```

Detects:

- Interactive access into running containers




---

### Docker Socket Exposure

```
index=main sourcetype=linux:audit
"key=\"docker_cmd\"" "type=EXECVE"
"/var/run/docker.sock:/var/run/docker.sock"
```

Detects:

- Containers with direct access to Docker daemon

This is a known and widely abused breakout vector.

---

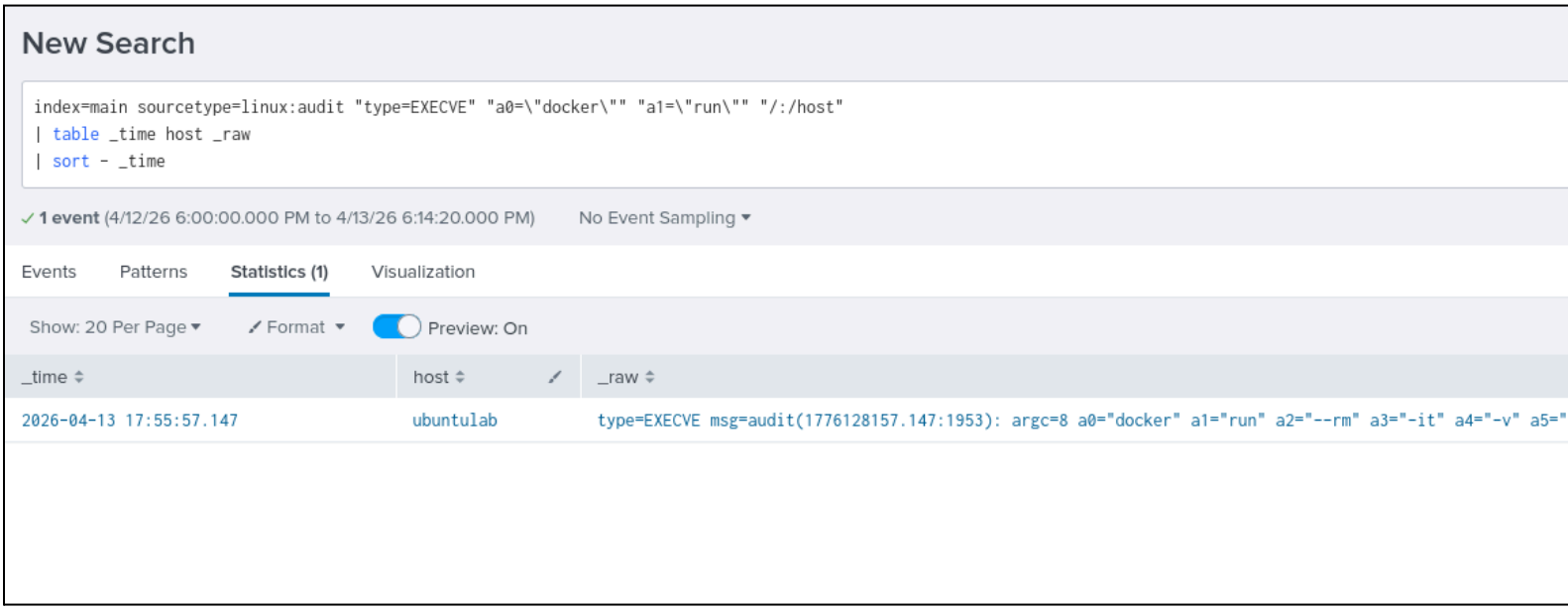
### Host Filesystem Mount (Critical)

```
index=main sourcetype=linux:audit
"key=\"docker_cmd\"" "type=EXECVE" "/:/host"
```

Detects:

- Mounting the entire host filesystem into a container

This is a **high-confidence indicator of compromise**.



---

### Privileged Container Execution

```
index=main sourcetype=linux:audit
"key=\"docker_cmd\"" "type=EXECVE" "--privileged"
```

Detects:

- Containers running with elevated privileges

---

### General Docker Activity

```
index=main sourcetype=linux:audit
"key=\"docker_cmd\"" "type=EXECVE" "a0=\"docker\""
```

Provides:

- Baseline visibility into Docker usage
- Useful for anomaly detection

---

## Zeek Network Telemetry (Limitations)

Zeek was meant to be used to analyze network activity during the attack window by reviewing:

- `conn.log`
- `dns.log`

While Zeek successfully captured traffic within the lab environment, most observed activity was:

- Internal (192.168.x.x)
- Between lab systems

Clear attribution of external container-generated traffic (e.g., DNS queries to `example.com`) was not consistently observable.

This is likely due to:

- Docker bridge/NAT behavior
- Interface monitoring limitations
- Virtualized network design

---

### Key Takeaway from Zeek

> Network telemetry alone was not sufficient to reliably attribute container activity in this environment.

---

## Key Insight

> Host-based telemetry (auditd) provided significantly stronger detection capability than container logs or network telemetry in this lab.

This reinforces an important principle:

> **When network visibility is limited, endpoint/host telemetry becomes the source of truth.**

---

## Findings

- Successfully detected Docker command execution using auditd
- Identified high-risk container configurations:
  - Docker socket exposure
  - Host filesystem mounts
  - Privileged containers
- Demonstrated realistic container breakout techniques
- Highlighted limitations of Docker logs and network visibility

---

## Limitations

- Docker JSON logs only capture stdout/stderr

- Splunk field extraction for audit logs was incomplete
- Zeek visibility limited by network architecture
- Container attribution is indirect without deeper instrumentation

---

## Future Improvements

- Implement Sysmon for Linux for richer process telemetry
- Add Docker daemon logs for lifecycle visibility
- Improve Zeek interface placement (monitor docker bridge)
- Explore Falco for runtime container security
- Build behavioral detections across time windows

---

## Closing Thoughts

This project ended up being less about Docker itself and more about understanding how to extract meaningful signal from imperfect telemetry. I did have to redo the lab due to the original design not allowing meaningful-enough data to show attacker actions, but that helped give an opportunity to engineer a better detection.

What worked:

- Host-based telemetry
- Direct observation of command execution
- Realistic attacker behavior

What didn’t:

- Relying solely on initial container logs
- Expecting perfect field extraction
- Forcing network attribution where visibility didn’t exist

That challenge of working with initial telemetry sources to thinking through real-world constraints - was the most valuable part of this lab.